

Contest Submission Source Code AI Evaluation Results Report

Chonnam National University PIMM Algorithm Party,
First Half of 2025

INFORMATION



Author	— Chonnam National University Game Development Club PIMM
Evaluation period	— 2025-03-30 (Sun) → 2025-04-05 (Sat)
Scope	— 1,035 code submissions to the <First Half of 2025 Chonnam National University PIMM Algorithm Party> contest

Introduction

AI is advancing rapidly. In just a year or two, it has become deeply embedded in our lives, completely changing our daily routines, schools, workplaces, and society.

Computer science in particular is one of the fields strongly affected by AI. Many computer science undergraduates, developers, and researchers work alongside AI. Programming has become such a major target of AI development that it is now used as a measure of AI performance. Starting with ChatGPT, various generative AI providers have expressed their models' performance in Codeforces ratings.

AI submissions are now easy to find in Codeforces contests. This is not limited to Codeforces: it is happening on competitive programming platforms everywhere, including Baekjoon Online Judge.

Some argue that the arrival of AI has made studying computational algorithms unnecessary.

Nevertheless, our algorithm contest team believes that human-written programs and competitive programming contests without AI have not lost their value. We therefore prohibited AI-generated solutions in the contest and invested considerable time in analyzing every submission.

We apologize for the delay.
Here are the results of our analysis.

Park Jonghyeon^{belline0124}, Chonnam National University Game Development Club PIMM.

This report is also available at <https://github.com/pimm-dev/2025-first-half-algorithm-party-editorial>.

Evaluation Results

We conclude that the following participants submitted solutions using generative AI:

- User #159 • User #161 • User #175
 - These participants repeatedly submitted solutions suspected of being AI-generated under various AI-generated code detection methods, or submitted solutions for which an AI-generated origin provides a sufficiently compelling explanation.
 - We excluded these participants from the scoreboard and prize eligibility.
 - We reported all submissions by these participants to Startlink and solved.ac as suspected cases of generative AI use.
 - We submitted to Startlink and solved.ac AI-generated solutions that were similar to these participants' solutions.
-

The following participants may have submitted solutions using generative AI:

- User #179 • User #077 • User #157
 - We reported all submissions by these participants to Startlink and solved.ac as suspected cases of generative AI use.
 - We submitted to Startlink and solved.ac AI-generated solutions that were similar to these participants' solutions.
-

- The numbers assigned to participants' masked IDs are based on the lexicographic ordering of MD5 hashes of their salted Baekjoon Online Judge handles. (The specific salting formula is not disclosed.)
- Participants who wish to find out their own masked ID should contact the club through the channels below.

Objections

Please submit any objections to this report through the club's contact channels within 30 days of publication.

Club Contact Channels

For inquiries about this report, please use the contact information below.

- Email — pimm.jnu@gmail.com
- Instagram — [@pimm.jnu](https://www.instagram.com/pimm.jnu)
- GitHub — [@pimm-dev](https://github.com/pimm-dev)

How We Conducted the Analysis

Method 1. Use AI-Code-Detector

Method 2. Generate solutions to contest problems with various AIs and compare similarity

Method 3. Apply a scoring formula to submission data to assess suspicion levels

Method 4. Check for comments widely associated with AI-generated code

Method 5. Manually review solutions suspected of being AI-generated based on the results of <Method 1>, <Method 2>, <Method 3>, and <Method 4>

Method 1. Use AI-Code-Detector

Procedure

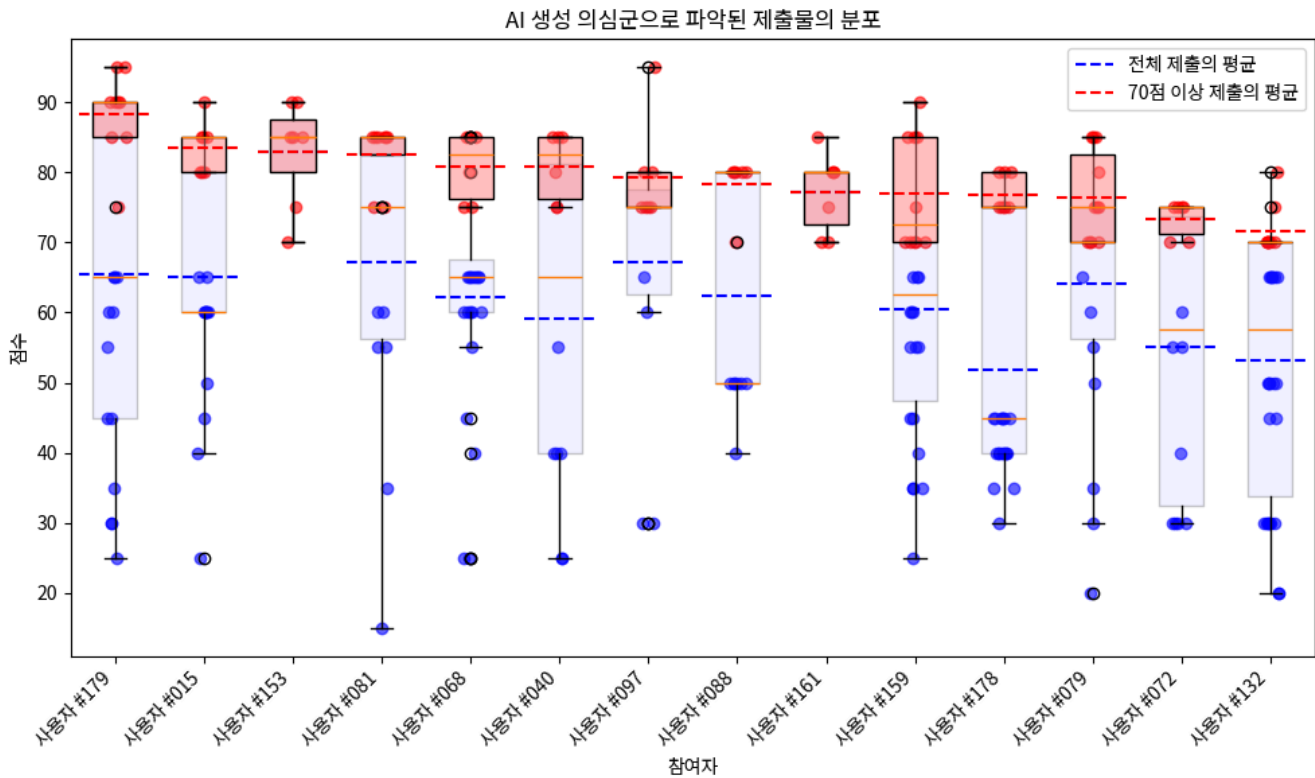
- This method uses an LLM fine-tuned to detect AI-generated code to assess whether code was generated by AI.
 - This evaluation used the AI-Code-Detector model available through Poe: <https://poe.com/AI-Code-Detector>
- A participant was judged likely to have generated and submitted code using AI if they had at least six submissions each rated at least 70% likely to be AI-written and the average rating of those submissions was at least 75%.
 - In view of the model's accuracy, submissions shorter than 350 characters were excluded from the evaluation.
- The following prompt was given to the model along with the source code:

Evaluate how much AI-generated probability they have using given source code files.
The result must be in CSV format and contain the filename and the calculated score.
If there are any significant viewpoints when evaluating code, give me a simple summary referring to the actual given source code.
- This evaluation is used as reference material: AI-Code-Detector was not fine-tuned specifically for algorithmic problem solving, and it has not been verified whether its assessment of the likelihood of AI generation is adequately explanatory.

Results

Participant	Mean score	Count	Submissions
User #088	78.33%	6	9225***5, 9225***7, 9225***3, 9225***7, 9225***4, 9225***2
User #159	77.00%	10	9225***6, 9225***6, 9225***8, 9225***0, 9225***0, 9225***8, 9225***3, 9225***1, 9226***0, 9226***0
User #179	88.33%	9	9225***9, 9225***9, 9225***0, 9225***2, 9225***0, 9225***8, 9225***4, 9225***4, 9226***7
User #068	80.83%	6	9225***2, 9225***2, 9225***4, 9225***7, 9225***2, 9225***7
User #040	80.83%	6	9225***9, 9225***7, 9225***4, 9225***2, 9226***0, 9226***9
User #081	82.50%	8	9225***4, 9225***5, 9225***4, 9225***5, 9225***3, 9225***1, 9225***2, 9225***6
User #072	73.33%	6	9225***4, 9225***0, 9225***8, 9225***3, 9225***2, 9225***6

User #153	82.86%	7	9225***5, 9225***1, 9225***0, 9225***5, 9225***6, 9225***7, 9225***8
User #097	79.29%	7	9225***4, 9225***9, 9225***8, 9225***7, 9225***3, 9225***9, 9225***2
User #132	71.67%	9	9225***4, 9225***1, 9225***6, 9225***6, 9225***6, 9225***7, 9225***7, 9225***7, 9225***5
User #161	77.14%	7	9225***0, 9225***0, 9225***2, 9225***9, 9225***5, 9225***5, 9225***1
User #178	76.88%	8	9225***9, 9225***6, 9225***5, 9225***6, 9225***3, 9225***6, 9225***2, 9225***2
User #015	83.57%	7	9225***7, 9225***7, 9225***0, 9225***2, 9225***0, 9225***0, 9225***2
User #079	76.36%	11	9225***2, 9225***1, 9225***1, 9225***5, 9225***9, 9225***8, 9225***6, 9225***3, 9226***5, 9226***6, 9226***7



In this table, some participants were incorrectly flagged as AI users (false positives) because of personal code templates, consistent formatting, and similar factors.

Types of code that AI-Code-Detector rated as more likely to be AI-generated:

- Extensive and consistent exception handling
- Consistent formatting[†]
- Advanced data structures
- Complex algorithm implementations[†]
- Systematic variable naming[†]
- Comprehensive modular organization

- Well-organized documentation patterns and comments
- Standard code structure[†]
- Template code[†]
- Use of regular expressions
- One-line code[†]
- Patterns of library use

[†]: Types also commonly found in human-written code for algorithmic problem solving

Types of code that AI-Code-Detector rated as less likely to be AI-generated:

- More irregular formatting
- Inconsistent naming conventions
- Less structured code
- Simpler logic flow
- A more direct/procedural approach[†]
- Basic exception handling[†]
- Fewer comments

[†]: Types also found in AI-generated code for algorithmic problem solving

- Submissions 9225***8, 9225***1, 9225***0, and 9225***9 were identified as representative examples of AI-generated one-line code.
- Submissions 9225***8, 9225***1, and 9225***1 were identified as representative examples of AI-generated code using regular expressions.
- In addition, (A) submission 9225***2 was identified as typical one-line code written with regular expressions, (B) submission 9225***5 as Rust code containing a complex system implementation, and (C) submission 9225***0 as AI-written because of extensible lambda expressions and import statements.
 - For (B) and (C), we confirmed that personal template code consistently present in all submissions caused the false-positive assessments.

Method 2. Generate contest solutions with various AIs and compare similarity

Procedure

1. Generate a comparison set of solutions using various AI models.
2. Encode the comparison set and contest submissions as embedding vectors using an AI model.
3. Compare the resulting embedding vectors.
4. Treat submissions with similarity of at least 97% as suspected AI-generated submissions.

1. In step 1, we generated the comparison set as follows.

- We used the following prompt to generate the comparison solutions with AI:

(Full contest problem statement)

Solve problem using C

Then, entering use cpp, use java, and use python in succession generated solutions in four languages in total

- We used the following AIs to generate the comparison set:

- | | | |
|--------------------|---------------------|---------------------|
| ▸ ChatGPT 4o | ▸ o1 mini | ▸ o3 mini |
| ▸ Claude 3.5 Haiku | ▸ Claude 3.5 Sonnet | ▸ Claude 3.7 Sonnet |
| ▸ Gemini 1.5 Flash | ▸ Gemini 1.5 Pro | ▸ Gemini 2.0 Flash |
| ▸ Deepseek V3 FW | ▸ Deepseek R1 | |
| ▸ Llama 3 70b Groq | ▸ Llama 3.3 70b FW | |
| ▸ Mistral Medium | ▸ Grok 2 | |

2. In step 2, we used the GraphCodeBERT model and pretrained parameters^{microsoft/graphcodebert-base} published on Hugging Face.
3. In step 3, we assessed the similarity between the two embedding vectors using cosine similarity.

- This evaluation is used as reference material: GraphCodeBERT was not fine-tuned specifically for algorithmic problem solving, and it has not been sufficiently verified whether the similarity-checking method is appropriate.

Results

Participant	Submission	Comparison	Similarity	Assessment
User #077	92259***0	f/llama-3.3-70b-fw	0.984	Accepted as no AI use ^{1,2}
	92259***3	f/llama-3.3-70b-fw	0.984	
	92260***2	f/llama-3.3-70b-fw	0.9835	
	92259***7	f/llama-3.3-70b-fw	0.984	
	92259***4	f/llama-3.3-70b-fw	0.984	
User #053	92258***9	f/llama-3.3-70b-fw	0.9727	False positive ³
	92259***5	f/llama-3.3-70b-fw	0.9707	
	92258***3	f/llama-3.3-70b-fw	0.9709	
	92259***0	f/llama-3.3-70b-fw	0.9709	
	92253***7	c/llama-3-70b-groq	0.9726	
	92253***7	c/mistral-medium	0.973	
	92253***7	c/gemini-2.0-flash	0.9702	
User #016	92256***0	c/llama-3-70b-groq	0.97	False positive ³
User #066	92255***7	c/deepseek-r1	0.9764	False positive ³
User #132	92256***7	c/deepseek-r1	0.9705	False positive ³
	92256***7	c/gemini-1.5-flash	0.971	
	92256***7	c/gemini-1.5-pro	0.9736	Accepted as no AI use ¹
	92255***6	c/deepseek-r1	0.9738	
	92255***6	c/gemini-1.5-flash	0.9727	False positive ³
	92255***6	c/gemini-1.5-pro	0.9766	
User #157	92252***7	c/o1-mini	0.9754	False positive ³
	92251***7	a/llama-3.3-70b-fw	0.9742	Written by AI
	92251***7	a/deepseek-v3-fw	0.9715	Accepted as no AI use ¹
User #023	92256***5	c/deepseek-r1	0.9769	False positive ³
	92256***5	c/gemini-1.5-flash	0.974	
	92256***5	c/gemini-1.5-pro	0.9742	
	92254***7	c/deepseek-r1	0.9762	
	92254***7	c/gemini-1.5-flash	0.9702	
	92254***5	c/deepseek-r1	0.9767	

	92254***5	c/gemini-1.5-flash	0.9717	
	92254***5	c/gemini-1.5-pro	0.972	
User #187	92258***4	c/deepseek-r1	0.9778	False positive ³
	92258***4	c/gemini-1.5-flash	0.9717	
	92254***2	b/grok-2	0.9728	
User #179	92251***9	c/gemini-1.5-flash	0.9727	False positive ³
User #188	92257***1	b/gpt-4o	0.9711	False positive ³
	92257***1	a/llama-3.3-70b-fw	0.9718	
User #131	92252***1	b/gpt-4o	0.9715	False positive ³
	92252***8	b/gpt-4o	0.973	
User #094	92256***9	b/mistral-medium	0.9707	False positive ³
User #105	92260***5	b/claude-3.5-sonnet	0.9713	False positive ³
	92260***5	b/gpt-4o	0.972	
User #046	92261***5	a/llama-3.3-70b-fw	0.97	False positive ³
User #058	92259***7	a/deepseek-v3-fw	0.9716	False positive ³
User #047	92257***7	a/llama-3.3-70b-fw	0.9727	Written by AI
User #029	92255***5	a/gemini-2.0-flash	0.9749	Accepted as no AI use ^{1,4}
	92255***5	a/deepseek-v3-fw	0.9746	False positive ³
User #140	92255***1	a/llama-3.3-70b-fw	0.9713	Written by AI
	92255***1	a/deepseek-v3-fw	0.9818	Written by AI
User #124	92254***6	a/llama-3.3-70b-fw	0.98	False positive ³
	92254***6	a/deepseek-v3-fw	0.9703	False positive ³
User #159	92251***6	a/deepseek-v3-fw	0.9811	Written by AI

1. AI-generated code is possible but not certain.

With no additional evidence to establish AI use, we accept that the participant wrote the code themselves.

2. AI generation is possible because the pattern below closely resembles a solution generated by the llama-3.3-70b-fw model and uses `throws IOException {}` in the program entry-point declaration, which is unusual in algorithmic problem solving.

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.util.*;

public class Main {
    static StringTokenizer st;
    static StringBuilder sb = new StringBuilder();
```

```
public static void main(String[] args) throws IOException {
```

3. No connection between the two code samples.

The embedding vectors generated by GraphCodeBERT simply appear to be similar.

4. The two code samples are similar in variable names, code structure, and function use, but their short length means the similarity may be coincidental.

Gemini 2.0 Flash	Submitted solution
<pre>#include <iostream> #include <string> using namespace std; int main() { int n; cin >> n; cin.ignore(); // Consume the newline character string slogan; getline(cin, slogan); int total_count = 0; string current_number = ""; for (char c : slogan) { if (c == '.' c == ' ' c == ':' c == '#') { total_count += stoi(current_number); current_number = ""; } else { current_number += c; } } total_count += stoi(current_number); cout << total_count << endl; return 0; }</pre>	<pre>#include <iostream> #include <string> using namespace std; int main() { int n; string slogan; cin >> n; cin >> slogan; int sum = 0; string current_number = ""; for (char c : slogan) { if (c == '.' c == ' ' c == ':' c == '#') { if (!current_number.empty()) { sum += stoi(current_number); current_number = ""; } } else { current_number += c; } } if (!current_number.empty()) { sum += stoi(current_number); } cout << sum; }</pre>

Method 3. Apply a scoring formula to submission data to assess suspicion levels

$$\text{score}(c_1, c_2) = \frac{w_{\Delta c} \text{len}(\Delta c)}{w_{\Delta t} \Delta t \cdot (w_l(c_1, c_2)^{1.5})}$$

c_1, c_2 = two successive submissions for a given problem

$\text{len}(\Delta c)$ = code length of diff c_1, c_2

$l(c)$ = language of submission c

Δt = unix time diff of c_1, c_2

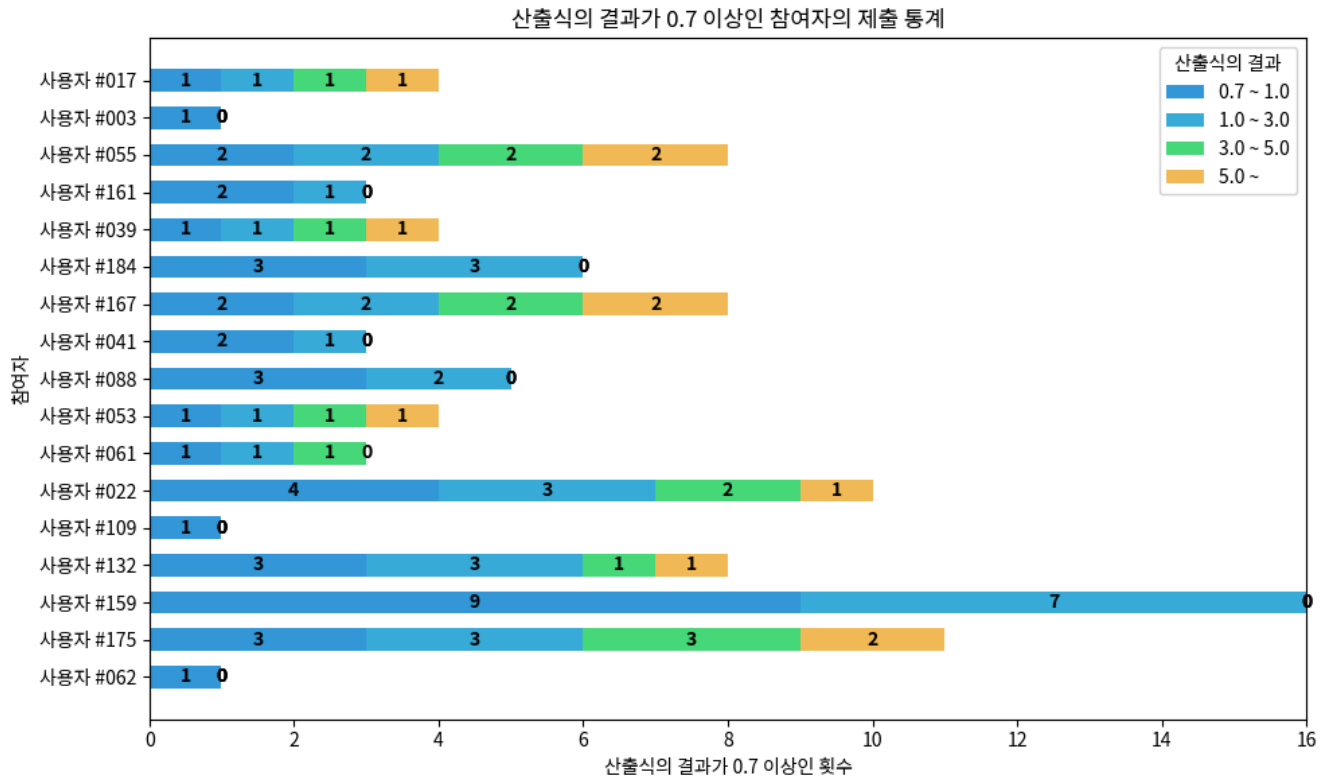
$w_l(c_1, c_2) = \{0.05 \text{ if } l(c_1) \neq l(c_2) 1.2 \text{ if } l(c_1) = \text{Java} 0.8 \text{ if } l(c_1) = \text{Python} 1 \text{ else}$

$w_{\Delta c} = 10$

$w_{\Delta t} = 6$

- This formula gives high scores in these cases:
 - Submitting solutions to the same problem in different languages
 - Submissions with substantial changes made within a short period
- This formula is applied to each participant's submission history for each problem.
 - If User #001 submitted three solutions to problem A, submissions #1, #2, and #3, we calculate $\text{score}(\text{submission \#1, submission \#2})$ and $\text{score}(\text{submission \#2, submission \#3})$.
 - If User #002 submitted only two solutions during the contest, one to problem A (submission #10) and one to problem B (submission #11), the formula cannot be applied to evaluate them.
- This evaluation is used as reference material: the suitability of the weights and each operator in this formula has not been sufficiently examined. Therefore, even many submissions with high calculated scores do not establish that a participant used AI.

Results



The analysis found that submissions with scores in $[0.7, 3.0]$ were slightly more likely to have used AI than submissions with scores outside this range.

Among the submissions that scored highly under this formula but were not written by AI was the following example:

Before (User #109, submission 9225***9)

```
//Could this really be brute force~~~~?

var [n,s] = require('fs').readFileSync(0,'utf8').trim().split('\n')
n = +n.split(' ')[0]
var min = n

// †: (rest of the submitted solution follows)

console.log(min)
```

After (User #109, submission 9225***2)

```
//Could this really be brute force~~~~?
//Wait, after thinking about it for an hour, the problem was a typo + misreading the input format.. How is
this possible (sob)
/*
```





```
// †: (the remaining ASCII art and solution follow)
```

†: These comments were not in the original submissions; the report author added them when excerpting the code.

Method 4. Check for comments and functions widely associated with AI-generated code

Procedure

- Check whether algorithmic problem-solving submissions contain comments widely known to be commonly generated by AI.
- Suspect AI generation if natural-language comments contain the following terms or features:

1. Output (= Print, Output)
2. Step (= Step)
3. Calculation
4. (Use of polite language)
5. Matches the regular expression `(//|#)(.*?):` (i.e. "Example:", "Note:")

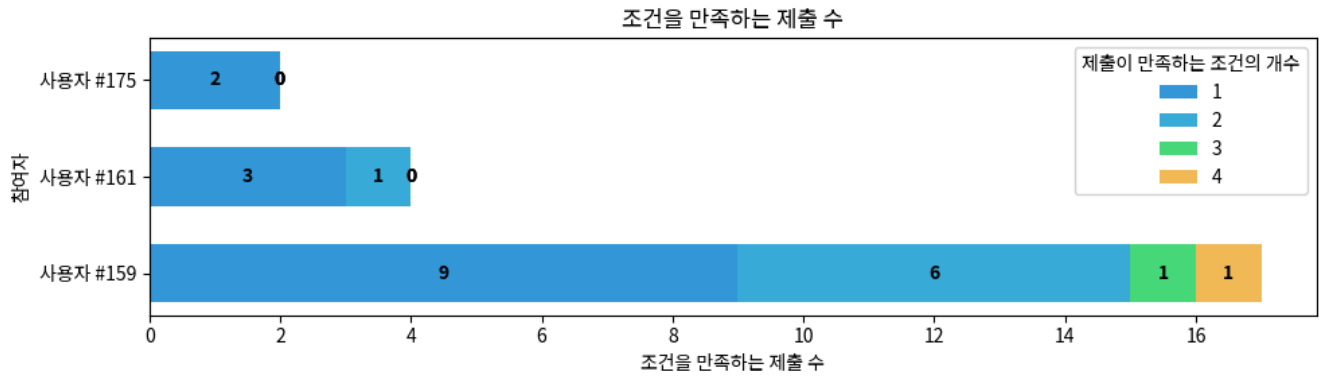
- Suspect AI generation if the following functions or modules are used:

6. C/C++: `free()`
7. Python: `re`[†]

[†]: The use of Python's `re` module was common in a typical kind of AI-generated code in this contest. However, it was already checked in <Method 1>, and is widely used in code golf, so it was excluded from this method.

Results

Participant	Count	Matching submissions (number of matching conditions)
User #175	2	9226***5(1), 9226***3(1)
User #161	4	9225***0(1), 9225***2(1), 9225***1(2), 9225***9(1)
User #159	17	9225***1(4), 9225***0(3), 9226***4(2), 9226***6(2), 9226***1(2), 9226***2(2), 9225***6(2), 9226***0(2), 9225***5(1), 9225***0(1), 9225***8(1), 9225***0(1), 9225***4(1), 9225***3(1), 9225***1(1), 9226***0(1), 9226***2(1)



At this stage, some submissions were falsely flagged as suspicious because of natural-language comments in personal template code, as in submission 9225***0:

User #050, submission 9225***0

```
// †: (submission code up to the preceding line)
/* read it as if you were wrong once. --> "why is this wrong??"

* basic strategy:
* don't be obsessed with speed or memory when the input is small compared
to limit
* internalization of problem statements
* simplify. a step-by-step approach
* readability is important
* Do I have to solve like this?

* stuff you should look for
* 0-based or 1-based?
* off-by-one error
* int overflow, array bounds (habituation of assert and debug)
* special cases (n=1?)
* do smth instead of nothing and stay organized
* WRITE STUFF DOWN
* DON'T GET STUCK ON ONE APPROACH (feat. BFS)

* after solving the problem
* consider whether there is another way.
* reduce memory, time, codes, ...
* what is my weakness that need to be addressed by solving this problem?
*/
```

- This comment consistently appears in User #050's regular submissions, so we considered it personal template code and removed the submission from the suspicious group.